HackInsight

Data-to-Decision

Participant Guide, Interactive Edition

From a real problem in Pakistan to a trustworthy data pipeline, an agentic AI system, and a project you can confidently explain to the jury.

Online | Teams of 4 students | 72 hours

Mon 19 Oct 2026, 12:00 AM to Wed 21 Oct 2026, 11:59 PM (PKT)

How to use this PDF
Blue underlined text is a clickable link, both inside the guide and to websites. Click any entry in the Contents, or 'Go to Contents' at the top of a page, to jump. Open the bookmarks panel of your PDF viewer to move between chapters. Tick the boxes and type in the blue fill-in fields. Use Adobe Acrobat, Chrome or Edge so that your answers are kept when you save the file.

Contents

1. Start here: what this event asks of you
2. Key dates and your 72-hour plan
3. How you are scored, and how to show each point
4. Analysing your chosen problem
5. Choosing and documenting your data
6. The warehouse and the make data command
7. The Data Quality Agent
8. The model: baseline, uncertainty and fairness by group
9. Optimization: better, faster, lighter, still reliable
10. The Insight Agent (RAG) and responsible AI
11. Automation, hourly runs and the dashboard
12. Business model: value, users and sustainability
13. Presenting to the jury
14. Rules of conduct and AI use
15. Final submission checklist

Each chapter also appears in the bookmarks panel. Type your answers in the blue boxes; they are saved in the file when you use Adobe Acrobat, Chrome or Edge.

1. Start here: what this event asks of you

HackInsight is a 72-hour online challenge for teams of four students. You choose one real problem that matters to people in Pakistan, gather trustworthy data about it from credible publishers, and turn that data into decisions: an automated pipeline, a model that beats a clear baseline, and two AI agents that keep the data honest and explain results with evidence.

Most data projects fail because of the data, not the model: unknown sources, no record of changes, test data leaking into training, and results nobody can reproduce. Generative AI also makes it easy to produce impressive output that is not true. This event rewards teams that can prove every statement they make.

The five things your team will deliver

#	Deliverable	In plain words
1	Data acquisition and warehouse	Scripts that fetch data from your sources and store it in three layers: raw, cleaned and processed. One command, make data, rebuilds everything.
2	Data Quality Agent	A program that runs validation tools on your data, decides to fix, fill or flag each problem, and explains every decision.
3	Model	A model that beats a baseline you declared in advance, with uncertainty shown and errors broken down by group.
4	Insight Agent (RAG)	An assistant that gives cited explanations for your primary user, and says "I don't know" when it has no support.
5	Automated pipeline and dashboard	A flow that runs every hour, plus a Hugging Face Space showing data freshness, quality status and the last successful run.

Around these five deliverables you should also be able to explain two things: the business model of your solution (who uses it, who pays, how it sustains itself) and how you optimized it for accuracy, speed and low resource use. They are covered in chapters 12 and 9.

Eight tracks to choose from

Track	Typical question a team might ask
Climate and heat-health	When do heatwaves become dangerous for outdoor workers in a city?
Air quality	Which days and areas will have unhealthy air, and who should be warned?
Energy demand	How much electricity will a region need tomorrow, and why does it change?
Public health	Where and when is a seasonal disease likely to rise, given weather and reports?
Food security	Which food prices are rising fastest, and which districts feel it most?
Education	Which students or schools need support to prevent dropout?
Information integrity	Which news claims are likely false, and how can readers verify them?
Urban mobility	Where will traffic or transit delays build up, and when?

You may also propose your own problem, as long as the data meets the same standard as the eight tracks (see chapter 5).

Go to Contents

2. How you are scored, and how to show each point

Criterion	Points	What the jury looks for	How to show it
Automatic checks	40	Improvement over your declared baseline, reproducibility, honesty of results.	Declare the baseline early, keep a clean make data, never hard-code outputs.
Dataset selection, integrity and strength	20	Credible sources, licences, documentation, checks, enough volume to be meaningful.	A complete manifest, a clear Dataset Declaration, and quality reports.
Agentic design and automation	15	Agents that really use tools and decide; a pipeline that runs on its own.	Show the agent's decision log and the Prefect run history.
Scientific rigour	10	Fair evaluation: no leakage, uncertainty, errors by group.	Time-based splits, intervals, per-group tables.
Usefulness and responsible AI	10	Value for the primary user; no invented numbers; cited claims.	Row IDs behind every number, citations, honest limits.
Viva and demo	5	Clear explanation and real understanding of the code.	Rehearsed talk; every member can answer questions.

Pakistan Impact Bonus (up to 10 extra points)

Part	Points	What it means
Problem	3	The problem is genuinely about people in Pakistan.
Data	3	The data reflects Pakistan: its places, seasons, institutions.
Local usability	2	A real person in Pakistan could use the output. Urdu output helps here.
Local validation	2	You checked results against a local source, report or expert.

Ties are broken by the higher bonus. The bonus cannot cancel a failed integrity check, and it is not given for simply attaching a Pakistani label to a problem that is not really about Pakistan.

Optional extras the jury may reward

- **Data Enrichment Agent:** an agent that finds and proposes extra credible sources.
- **Urdu output:** explanations or dashboard text in Urdu.
- **Supervisor agent:** one agent that coordinates the others.
- **Drift monitor:** detects when new data stops looking like the data you trained on.
- **Human-in-the-loop approval:** a person approves risky agent decisions.
- **Change Request 2:** respond to any change request posted by the organisers.

3. Analysing your chosen problem

The jury will not only look at your code. They will ask: Why this problem? Who is it for? How do you know it is real? What would change because of your work? A clear problem analysis gives you the answers, and it guides every technical choice you make later.

The seven-step problem analysis

1. **Name the primary user and the decision.** Who will use your output, and what will they decide differently? Example users: a district health officer, a grid planner, a school head teacher, a market price monitor. If you cannot name a person and a decision, the problem is still too vague.
2. **Describe the problem with evidence.** What is happening, where, and since when? Support it with official statistics, reports or notices from credible publishers, and cite them.
3. **Say who is affected and where.** Which groups (age, income, occupation, gender) and which places (city, district, province)? This also tells you which groups to check for unequal model errors later.
4. **Define the outcome you will predict or explain.** Be precise: what is measured, how often, and in which unit. This becomes your target variable or your analysis question.
5. **Set a baseline and a success measure.** What is the simplest sensible answer (for example, "tomorrow equals today")? How much better must you be for it to matter to the user?
6. **List limits and risks.** Missing data, biased coverage, delays in reporting, places with no sensors. Say what your data cannot tell you.
7. **State your recommendation, and what you will not claim.** Outputs are information, not professional advice. Say clearly where a user should go to an official service instead.

Worksheet: write your problem analysis

Fill these fields now. Review them again before the viva, they are your talking notes.

Team name

Spark H

Chosen track or own problem

Climate and heat-health (disaster risk), with a secondary food-security model

Problem statement (one or two sentences)

District planners in Pakistan get provincial rain alerts but no district-specific, calibrated probability of unusually heavy rain 1-3 days ahead. FloodLens gives one for all 160 districts, refreshed every hour.

Primary user and the decision they make

A district / PDMA disaster-management planner deciding, 24-72 h ahead, where to pre-position rescue teams, pumps and relief stock and where to issue local warnings.

Why this matters in Pakistan (with a cited source)

The 2025 monsoon killed 1,037 people and injured 1,067 between 26 Jun and 1 Oct 2025, 509 of them in KP (NDMA, Monsoon 2025 Final Consolidated Situation Report No. 87, 2 Oct 2025).

Who is affected and where (groups and places)

All 160 districts (OCHA admin-2); errors reported by province, mountain vs plain, coastal vs inland, 2022 flood-prone vs not, and monsoon vs dry months.

Outcome you will predict or explain (what, how often, unit)

Probability that a district's daily wettest-pixel rain (NASA IMERG Late, UTC day) exceeds its own 95th percentile of wet-day rain (2015-2023), for +1, +2 and +3 days; binary, daily, per district.

Declared baseline and what counts as better

District-by-month climatology (share of 2015-2023 days over the threshold). Better = lower Brier score at 24 h on the test set, and the gain must also hold in monsoon months.

Limits and risks you already see

Daily 0.1 deg satellite rain smooths out local cloudbursts (Buner, 15 Aug 2025); forecasts are taken at district centroids; forecast archive starts Feb 2024 (17 months to train); heavy rain is not a flood.

4. Choosing and documenting your data

No data is provided. Each team selects credible sources, justifies the choice, and gathers the data itself during the event, using the source's licence and terms of use. Your job is to show that you can trust every row you use and to explain where it came from.

Rules that apply to your data

Requirement	Detail
Independent sources	At least 2 independent sources, including at least 1 credible source that is not on the track list.
Minimum volume	At least 50,000 rows, or 2 years of hourly data, or 5,000 labelled documents or images.
Gathered during the event	Use only data your team gathered during the event, under its licence and terms of use.
Manifest	Record every raw file in manifest.csv: source, query, fetch time, SHA-256 and licence.
Enough to work on	The data must be large and messy enough that there is real engineering to do.

Not allowed

- Files whose only source is a re-upload on Kaggle, GitHub or a blog, with no original publisher.
- Synthetic or AI-generated data for training or evaluation. (Clearly labelled stress tests only.)
- Personal, patient or other identifiable data.
- Data collected against a site's terms of use or robots.txt.
- Data so small or so clean that there is nothing to engineer.

Where to look

Track	Real-time sources	Benchmark sources
Climate and heat-health	Open-Meteo, Meteostat, NASA POWER	ERA5 (Copernicus), MODIS land surface temperature
Air quality	OpenAQ (free key), Copernicus CAMS	Beijing Multi-Site Air Quality (UCI)
Energy demand	ENTSO-E Transparency Platform, Open-Meteo	UCI Electricity Load Diagrams; ETT
Public health	Weather APIs, health department case reports	OpenDengue
Food security	WFP food prices on HDX	FAOSTAT price and production series
Education	Public education statistics	OULAD; UCI Student Dropout
Information integrity	GDELT, news RSS	LIAR; FakeNewsNet; Urdu fake news corpus
Urban mobility	GTFS-Realtime feeds	METR-LA, PEMS-BAY

Pakistani sources to check first

Confirm coverage and licence before you rely on them.

Publisher	Link	Publisher	Link
Pakistan Bureau of Statistics	pbs.gov.pk	Pakistan Meteorological Department	pmd.gov.pk
National Disaster Management Authority	ndma.gov.pk	State Bank of Pakistan	sbp.org.pk
NEPRA (power regulator)	nepra.org.pk	Humanitarian Data Exchange (Pakistan datasets)	data.humdata.org
Open-Meteo (weather API)	open-meteo.com	OpenAQ (air quality)	openaq.org
NASA POWER	power.larc.nasa.gov		

Also check provincial health, education and environment departments. Some sites need a free account or key, and each source updates at its own speed, so test access early.

How to judge a source: a quick checklist

- ☐ Original publisher is named, and you can open the page.
- ☐ Licence allows your use, and terms of use and robots.txt allow your method of access.
- ☐ Covers Pakistan and the places and period your problem needs.
- ☐ Update frequency fits an hourly pipeline (or you have a clear replay plan).
- ☐ Fields and units are documented, so you can explain every column.
- ☐ It is independent of your other source (not just a copy of it).

Together, your sources reach the minimum volume.

Dataset Declaration (due Mon 19 Oct, 12:00 PM PKT)

The declaration is short. It states which sources you chose and why. Draft it here, then commit it to your repo before the deadline.

Source 1: name, publisher, URL or DOI, licence, update frequency

Open-Meteo Previous Runs API (forecasts issued 1-3 days ahead from ECMWF/NOAA/DWD models), Open-Meteo, open-meteo.com/en/docs/previous-runs-api, CC BY 4.0, model runs every 6 h.

Source 2: name, publisher, URL or DOI, licence, update frequency

NASA GPM IMERG V07 Late Run daily (GPM_3IMERGDL), NASA GES DISC, disc.gsfc.nasa.gov, NASA EOSDIS open data policy, daily (~14 h latency).

Which source is not on the track list, and why is it credible?

NASA IMERG: NASA's standard multi-satellite rainfall product, independent of the weather models used as inputs. Also PBS weekly SPI, Pakistan's official weekly consumer prices.

Expected volume (rows, years of hourly data, or labelled items)

Forecasts 476,640 district-day-lead rows; IMERG 689,120 district-days (2015 to Oct 2026); PBS prices 146,523 rows (169 weeks).

Why this data fits the problem and the primary user

Archived forecasts give real 1-3 day lead time without leaking the future; satellite rain shows what fell even where there is no rain gauge; both cover every district the planner is responsible for.

Baseline you declare (must be fixed before you see results)

Model A: district-by-month climatology. Model B: no change (next week's price = this week's).

5. The warehouse and the make data command

A reliable project can be rebuilt by anyone. That is what reproducibility means, and it is worth a large share of the automatic checks.

Three layers

Layer	Contents	Rule of thumb
Raw	Files exactly as fetched, plus the manifest entry for each (source, query, fetch time, SHA-256, licence).	Never edit raw files. Treat them as evidence.
Cleaned	Types fixed, units unified, time zones aligned, duplicates handled, problems flagged.	Every change comes from code, never by hand.
Processed	Tables ready for analysis and modelling (the "gold" tables), with stable row IDs.	Every number you show must come from here.

Use DuckDB or SQLite as your warehouse: both are free, fast and need no server. Tools for fetching include requests, httpx, pdfplumber (for PDF reports) and Tesseract OCR (for scanned documents).

What make data must do

1. Start from an empty working folder.
2. Fetch or reload every raw file and verify its SHA-256 against the manifest.
3. Build the cleaned layer.
4. Build the processed layer.
5. Run the quality checks and print a short summary.
6. Finish without any manual step.

Common data traps (and how to avoid them)

Trap	What goes wrong	Prevention
Time zones	Hourly records shift by 5 hours (UTC vs PKT) and merges fail silently.	Store UTC, convert to PKT only for display.
Mixed units	Temperatures in Kelvin and Celsius, prices in different currencies.	Write unit checks into validation.
Duplicate or late records	Reports are corrected later and counted twice.	Use unique keys and keep fetch time.
Leakage	Future information leaks into training.	Split by time; fit scalers only on training data.
Silent gaps	A sensor is offline and a gap looks like a zero.	Keep missing as missing and flag it.

6. The Data Quality Agent

The Data Quality Agent is not just a script of checks. It uses validation tools, decides what to do about each problem, and explains the decision. The explanation is what the jury will read, so make it specific.

How it works

1. **Run tools.** Use pandera or Great Expectations to test schema, ranges, missing values, duplicates, timeliness and distribution changes.
2. **Decide.** For each finding choose one action: fix (a safe, certain correction), fill (an estimate, clearly marked) or flag (leave as is and warn).
3. **Explain.** Write a short reason, the rule that triggered it, and how many rows were affected.
4. **Log.** Store the decision with a timestamp so it can be audited.

Choosing between fix, fill and flag

Situation	Suggested action	Why
Wrong data type or an obvious unit error with a known conversion	Fix	Certain and reversible.
A short gap in a smooth hourly series	Fill (mark the value as filled)	A reasonable estimate, but it must stay visible.
A long gap, or a value that looks impossible but cannot be confirmed	Flag	Do not guess. Let the user see the uncertainty.
A source stopped updating	Flag and show staleness on the dashboard	The user must know the data is old.

What one logged decision might look like

Write one real decision from your own quality log here, with the rule, the action and the row count.

Example logged decision (from your quality_log)

spi_price | rule spi_price_collected | action FLAG | 1,279 rows | reason: PBS published 0 or nothing for this city and item (not collected); kept NULL, never filled | decided 2026-10-21, replayed on every rebuild. Others: spi_min_le_avg_le_max FLAG 822 rows; spi_weekly_jump FLAG 399 rows; rain_district_day_timely FLAG (stale since 16 Oct).

7. The model: baseline, uncertainty and fairness by group

A model is only useful if it beats something simple, and if you can say how sure it is and where it fails.

Step 1: declare a baseline early

Write your baseline in the Dataset Declaration and do not change it after seeing results. Good baselines are simple: yesterday's value, the same hour last week, the average for each group, or a plain logistic or linear regression.

Step 2: split the data honestly

- Split by time: train on earlier periods, test on later ones.
- Keep the test set untouched until the very end.
- Fit scalers, encoders and imputers on training data only.
- Never tune the model on the test set.

Step 3: choose a model that you can explain

Scikit-learn, gradient boosting, Prophet and transformers are all suitable. Prefer the simplest model that clearly beats the baseline, because you must explain it in the viva.

Step 4: show uncertainty

Report a range, not just one number: prediction intervals, quantile models or bootstrapping. Explain in plain words what the range means for the user's decision.

Step 5: report errors by group

Possible group	Why it matters
Province or district	Some places have fewer sensors or reports.
Urban and rural	Coverage and quality often differ.
Season or time of day	Models often fail in extremes.
Age, gender or school type (for education and health)	Unequal errors can cause unfair decisions.

A single overall score can hide a group where the model performs badly. Show the table, name the weakest group, and say what a user should do about it.

Your baseline and model score (fill in after the test set is run once)

TEST (Jan-Sep 2026, run once): climatology 0.0192 [0.0156, 0.0231] vs model 0.0177 [0.0146, 0.0213] Brier at 24 h; paired diff -0.0014 [-0.0022, -0.0008]; monsoon 0.0251 vs 0.0237. Validation 0.0249 vs 0.0233. Weakest group: Azad Kashmir.

8. Optimization: better, faster, lighter, still reliable

Optimization means improving your solution on four fronts at once: how well it performs and how accurate it is, how fast it runs, how little computing it needs, and how much it costs to operate. The goal is not the highest score at any price. The goal is a solution that stays reliable while it becomes more efficient.

Rules of the road

1. **Get a working baseline first.** Optimize only something that already runs end to end.

2. **Measure before you change anything.** Record accuracy, run time, memory and API calls for the current version.
3. **Change one thing at a time** so you know what helped.
4. **Keep the test set untouched.** Tune on validation data only, using time-based splits.
5. **Check reliability after every change:** same seed gives the same result, no new leakage, and no group got worse.
6. **Stop when the gain is no longer worth the effort.** Say so in your notes.

Where you can optimize

Goal	Ideas to try	How to measure
Accuracy and quality	Better features (lags, rolling averages, seasonality), time-aware hyperparameter tuning, regularisation, simple ensembles, calibrated probabilities.	Your chosen metric vs the baseline, plus errors by group.
Speed	Vectorise instead of looping, profile slow steps, let DuckDB do heavy aggregation, cache results, fetch only new data instead of re-downloading everything.	Seconds per pipeline run and per step.
Memory	Use smaller data types, read files in chunks, store tables as Parquet, load only the columns you need.	Peak memory in MB.
Compute and energy	Prefer the smallest model that works, stop training early, use a smaller embedding model, cache embeddings, batch requests, limit calls to large language models.	CPU or GPU time, number of API calls, tokens used.
Pipeline efficiency	Skip steps whose inputs did not change, use incremental loads, set sensible retries so failures do not waste whole runs.	Run duration, failed or repeated runs.

Keeping results reliable while you optimize

- [x] Random seeds are fixed, and re-running gives the same numbers (or a stated small variation).
- [x] The same time-based split is used before and after each change.
- [ ] Accuracy did not drop for any important group (province, district, season).
- [x] Uncertainty ranges are still reported and still make sense.
- [x] Faster or lighter versions were compared against the full version on the same data.

Optimization log

Keep a short log as you go. It is excellent evidence in the viva. For each change note: what you changed, the metric before and after, the run time and memory before and after, and whether you kept it.

Change 1: what changed, before and after (accuracy, time, memory), kept or rejected

Logistic regression -> gradient boosting (both calibrated on validation): val Brier 24 h 0.0238 -> 0.0233; training 2.9 s -> 3.4 s; ~600 MB peak both. Kept.

Change 2: what changed, before and after (accuracy, time, memory), kept or rejected

IMERG cropped server-side (OPeNDAP) instead of global daily files: 33.3 MB -> ~74 KB per day; 4,307 days = 182 MB instead of ~143 GB; same values checked. Kept.

Change 3: what changed, before and after (accuracy, time, memory), kept or rejected

PBS PDF tables parsed once and cached by SHA-256: price build 286 s (380 MB) -> 3.6 s on rebuild, identical tables. Kept. (Model B quantile GBM: MAE 1.667 vs 1.665 no change: rejected.)

How to talk about a trade-off

Your trade-off in one or two sentences (what you gained, what it cost)

Gradient boosting gained 0.0005 Brier over logistic regression for 0.5 s more training and a less transparent model; kept because it also gained in monsoon months and in every group.

9. The Insight Agent (RAG) and responsible AI

RAG stands for retrieval-augmented generation: the agent first retrieves trusted documents and table rows, then writes an explanation that is based only on them. It is the part of the project most visible to a user, so honesty matters most here.

The three hard rules

Rule	In practice
1. No number from the LLM	Every number comes from a gold table. The insight stores the IDs of the rows it used.
2. No claim without a source	Every recommendation cites a trusted document and section (for example a health or weather advisory).
3. No synthetic data in evaluation	Test the agent on real data only. Simulated tests must be labelled as such.

How to build it

1. Collect a small library of trusted documents for your user: official guidelines, advisories and public reports.
2. Split them into sections and store them in Chroma or FAISS with multilingual embeddings.
3. When asked a question, retrieve the best sections and the matching gold-table rows.
4. Have the language model write only the wording. Numbers are inserted from the table by code.
5. If nothing relevant is found, the agent answers "I don't know" and says what is missing.

Notes on what a good answer contains

Write the parts a good answer should always include (section cited, row IDs, limits)

The quoted sentence with document title, page and section id; every number with its gold-table row id; "I don't know" below the retrieval threshold; "not an official warning"; Rescue 1122 / NDMA first in emergencies.

Tests you should run before the viva

- [x] Ask a question your documents can answer: it cites the right section.
- [x] Ask a question your documents cannot answer: it says "I don't know".
- [x] Check that every number in an answer matches the stored row IDs.
- [x] Try a misleading question: it does not invent a fact to please the user.

Health-related outputs point to official services in an emergency.

10. Automation, hourly runs and the dashboard

Your pipeline must run on its own every hour. Prefect is recommended; Airflow and Dagster are also allowed. The run history from Tue 4 PM to Wed 4 PM is part of your evidence.

A starter flow

The schedule `5 * * * *` means "at five minutes past every hour".

Dashboard on a Hugging Face Space

Build a simple Gradio or Streamlit app and publish it as a Hugging Face Space. It must show three things clearly:

- **Data freshness:** when each source was last fetched, and whether it is late.
- **Quality status:** the latest result of the Data Quality Agent.
- **Last successful run:** time and status of the latest pipeline run.

Learn more about the tools: Prefect | Gradio | Streamlit | DuckDB | pandera | Great Expectations

Run-day checklist

- [ ] The scheduler is running on a machine or service that will stay on for the full window.
- [x] Secrets and API keys are stored outside the code and outside the repo.
- [x] Retries are set, and a failed run does not corrupt earlier data.
- [x] The dashboard updates from the latest run, not from fixed numbers.

Export the run history as **prefect_runs.csv** for Tue 4 PM to Wed 4 PM, and take a dashboard screenshot.

11. Business model: value, users and sustainability

A good solution is not only accurate, it is used and kept running. The jury will want to hear how your AI or machine learning solution creates value, who will use it, and how it could pay for itself through earning revenue or reducing costs. You are not writing a full company plan. You are showing that you have thought about real people and real constraints.

Five questions to answer

1. **Who is the user?** The person who uses the output to make a decision.
2. **Who is the customer or payer?** The person or organisation that pays or provides the budget. This is often different from the user.
3. **What is the value?** What gets better: time saved, money saved, harm avoided, better decisions, fairer access?
4. **How does it sustain itself?** Revenue, cost reduction, a grant or a public budget.
5. **What does it cost to run?** Computing, storage, APIs, data licences, maintenance and people.

Write your value proposition

Complete this sentence: "For [user], who needs to [decision or task], our solution [what it does], so that [benefit], unlike [today's way of doing it]."

Ways a solution can sustain itself

Model	How it works	Good fit when
Subscription or licence	An organisation pays regularly for access to the dashboard or alerts.	Users are institutions with ongoing needs (utilities, hospitals, schools, city offices).
Pay per use	Charge per query, report or alert through an API.	Many different small users need occasional answers.
Contract, grant or public budget	A government body, NGO or funder pays for the service.	The benefit is public (health, safety, food security).
Free basic, paid advanced	Core information is free; extra features such as detailed alerts or reports are paid.	You want wide public benefit and a path to funding.
Cost reduction (internal use)	An organisation saves money or staff time by replacing manual work or preventing losses.	The main value is efficiency, not selling a product.

Cost reduction and value: how to reason about it

- **Time saved:** hours of manual work removed multiplied by the cost of an hour.
- **Losses avoided:** fewer wasted resources, earlier warnings, fewer emergency responses.
- **Better targeting:** help reaches the places and people who need it most.
- **Operating cost:** hosting and compute, API fees, storage, data licences and maintenance time.

A simple way to think about it: **net benefit = value created (revenue or savings) minus the cost of running the solution.** Compare it across a month or a year.

Risks to mention

- Users may not trust or adopt the tool, so explain how you would earn trust.
- Data sources may change, close or change licence terms.
- Model quality may drift over time and needs monitoring.
- Wrong outputs can harm people, especially in health and safety. Outputs are information, not professional advice.

Worksheet: your business model

Primary user and the decision they make

District / PDMA planner: where to pre-position teams and relief stock 1-3 days before heavy rain.

Customer or payer (who provides the budget)

PDMAs / NDMA (provincial preparedness budgets) or humanitarian donors funding anticipatory action.

Value proposition (one sentence)

For district disaster planners, who must decide where to pre-position teams before heavy rain, FloodLens gives an hourly, calibrated district rain-risk 1-3 days ahead with its evidence, unlike provincial alerts.

Revenue or cost-reduction model, and why it fits

Free public dashboard; contract or grant for PDMAs (alerts, support); triggers for donors' anticipatory action. Fits because the benefit is public safety and the cost is tiny.

Main running costs (compute, APIs, data, people) with estimates

Measured: 1 CPU; ~28 min rate-limited download + ~5 min rebuild per hour; 1.3 GB storage (+10 MB/day); NASA/OCHA/PBS data free; Open-Meteo free non-commercial (paid if commercial); host free or ~EUR 4/month.

Key assumptions behind your figures (and where they come from)

Free-tier terms of Open-Meteo and NASA stay as they are (their published terms); PDMA staff check a dashboard daily in monsoon; costs measured on our own runs during the event.

Biggest risks and how you would reduce them

Trust (citations, row ids, stated limits); a source blocking access (gpm1 blocked us on Wed; logged, switched to NASA cloud); drift (hourly quality checks); rain risk mistaken for a flood warning (labelled).

Go to Contents

Plan for the 3-minute demo video

Time	Show
0:00 to 0:30	The problem, the primary user and the decision.
0:30 to 1:15	The data sources and the quality decisions.
1:15 to 2:15	A live walk-through: dashboard, model vs baseline, one cited answer, one "I don't know".
2:15 to 3:00	Results by group, limits, and the Pakistani impact.

Questions to practise

Write a one-sentence answer for each. Practise saying them aloud.

Why did you choose this problem, and who is the user?

The 2025 monsoon killed 1,037 people (NDMA); our user is the district / PDMA planner who must pre-position teams.

How do you know the problem is real in Pakistan? What evidence did you use?

NDMA's own 87 monsoon 2025 sitreps: 125 districts named, 1,037 deaths in the final report (No. 87).

Why these sources? What licence applies and who is the original publisher?

Archived forecasts (Open-Meteo, CC BY 4.0, ECMWF/NOAA/DWD) for honest lead time; NASA IMERG (open data) as independent truth.

What was the worst problem in your raw data and how did you handle it?

NASA's gpm1 server blocked our IP and daily files are 33 MB; we switched to NASA's cloud OPeNDAP, cropped to ~74 KB/day.

Why did the Data Quality Agent flag instead of fill in one case?

A price PBS did not collect is unknown; filling it invents data, so 1,279 prices stay NULL and flagged.

What is your baseline, and by how much does your model beat it?

Test set: climatology 0.0192 vs model 0.0177 Brier at 24 h (~8% better, paired interval excludes zero; holds in monsoon).

Where does your model perform worst, and who might be affected?

Azad Kashmir (smallest gain); single-valley cloudbursts like Buner 15 Aug 2025 are missed by the satellite labels.

How do you make sure the Insight Agent never invents a number?

No model writes numbers: code inserts them from gold tables with row ids; text is quoted with section ids.

Who will use your solution, who would pay for it, and how could it sustain itself?

PDMA planners use it; PDMAs or donors pay from preparedness / anticipatory-action budgets; it runs on one small server.

What did you optimize, and what did it cost in accuracy, speed or resources?

Server-side cropping (143 GB -> 182 MB), a parse cache (286 s -> 3.6 s), gradient boosting (+0.0005 Brier) at some explainability.

12. Rules of conduct and AI use

- Outputs are information, not professional advice. Health outputs must point users to official services in an emergency.
- AI assistants are allowed. Disclose material use in AI_Use_Log.md and be able to explain all your code.
- Acknowledge all external models, APIs, libraries, datasets and code, and follow their licences and rate limits.
- Do not share code, data or outputs with other teams during the event.
- Label anything simulated or unfinished.
- Do not use personal or identifiable data of any kind.

What to write in AI_Use_Log.md

Entry	Example of what to note
Tool and version	Name of the assistant or model you used.
Task	Which part of the project it helped with (for example, drafting a cleaning function).
What you kept or changed	How you checked and edited the output.
Who can explain it	The team member who understands that code.

13. Final submission checklist

Tick each item as you finish it. The form shuts at 12:00 AM Thu 22 Oct (PKT).

What to submit

- [ ] GitHub repo tagged at the v1.0-final commit (nothing pushed after the tag is judged).
- [ ] Hugging Face Space link that opens and shows freshness, quality status and last successful run.
- [ ] prefect_runs.csv for Tue 20 Oct 4 PM to Wed 21 Oct 4 PM, unedited, plus a Prefect dashboard screenshot.
- [ ] Summary.pdf: one page only.
- [ ] Demo video: 3 minutes or less, link opens without a login.
- [ ] AI_Use_Log.md: honest and complete.

Physical viva: every member is available. Watch the official channel for the schedule.

Quality checks before you tag

- [ ] make data rebuilds raw, cleaned and processed layers from scratch.
- [ ] manifest.csv lists every raw file with source, query, fetch time, SHA-256 and licence.
- [ ] Dataset Declaration was committed before Mon 12:00 PM.
- [ ] Baseline was declared before results, and the model beats it.
- [ ] Uncertainty and errors by group are shown.
- [ ] Every number in an insight comes from a gold table, and row IDs are stored.
- [ ] Every recommendation cites a source. Unsupported questions get "I don't know".
- [ ] Anything simulated or unfinished is clearly labelled.
- [ ] A short business model (user, payer, value, running cost) is ready to explain.
- [ ] Your optimization log shows what you changed, what it gained and what it cost.
- [ ] Every team member can explain the problem, the data, the code and the results.

Disqualification

Hard-coded outputs, faked logs, leaked test data or misrepresentation.

Go to Contents